COMP 018: Database Administration - Activity 4 Progress Report
Access Control, Database Users, Roles, Privileges, and Security Governance

Project Identification & Metadata

Course	: COMP 018 - Database Administration
Evaluating Group	: Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository	: https://github.com/Nycri/brewpos
Target Project Monitored	: PUP-SPOT: Campus Lost and Found System
Target Project Group	: Group 5 (Imperial, Manansala, Napeñas, Lctz, Quody)
Target Project Repository	: https://github.com/grp05-dba/grp05_lostandfound
Primary Database	: grp05_lostandfounddb (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	: grp05_lostandfounddb_restore_test

ACTIVITY 4: Database Users, Roles, Privileges, and Access Control

1.1 Access-Control Plan & Architecture

In default database deployment configurations, applications frequently connect using an unrestricted superuser account. For PUP-SPOT, connecting interactive public portals and administrative tools using a database superuser account severely violates the Principle of Least Privilege. If an unauthenticated user were to exploit an application vulnerability, they would possess unrestricted data definition and modification capabilities across the cluster, threatening lost item histories, claim logs, and system integrity.

To establish secure operations for Group 5's PUP-SPOT system, a role-based access control architecture was implemented. This architecture separates staff administrative maintenance from operational traffic while strictly isolating public student workflows from administrative capabilities. The structure utilizes group roles without login capabilities to manage privilege collections, alongside individual login accounts assigned to matching group roles.

Under this access control architecture, the staff administrative runtime account is granted selection, insertion, and update capabilities across all operational tables, while confining record deletion strictly to automated match entries and image attachments. Row deletion remains strictly denied on core lost items, lost reports, claim records, and categories. Public student users are granted query access to browse items and categories, insertion rights for submitting lost reports and claims, status update capabilities on their active lost reports, and deletion rights strictly for potential match entries. Public accounts are prohibited from modifying catalog item details, altering claim decisions, or deleting submitted records. Logical database backups are delegated to a dedicated backup role with schema-wide selection rights and zero write permissions.

1.2 Setup Execution & Access-Control Hierarchy

The configuration process established three non-login group roles representing administrative runtime, public user, and backup profiles, followed by the creation of matching login accounts with dedicated authentication credentials. Access permissions were revoked from public roles across the database and schema to enforce explicit privilege assignment. Connection and schema usage rights were assigned to the designated group roles, specific table grants were applied according to operational requirements, and default schema privileges were configured to maintain access definitions for future objects.

1.3 Verification Results: Staff Admin Runtime Account Testing
Verification testing demonstrated that the staff administrative account successfully reads registered item records, inserts new catalog items while advancing sequence counters, updates item statuses, and deletes misfiled image attachments. When deletion operations were attempted on claim records, system categories, or underlying table structures, the database engine denied execution under the narrow deletion policy and object ownership constraints.

1.4 Verification Results: Public User Account Testing
Inspection testing confirmed that the public user account successfully executes query searches across found items and categories, and submits new lost reports while advancing sequence counters. Attempted modification operations, including updating catalog item statuses, deleting items, and inserting unauthorized category entries, were uniformly blocked by the database engine.

1.5 Verification Results: Cross-Database & Administrative Boundary Testing
Administrative isolation testing showed that database creation commands attempted under application accounts were denied due to missing creation privileges. Connection attempts to unrelated databases on the server instance failed because public connection permissions were revoked, restricting access strictly to authorized targets.

1.6 Issues, Fixes, and Feedback to Owners
During initial setup, default superuser configuration properties were replaced with least-privilege role credentials aligned with application environments. Routine maintenance scripts were adjusted to execute statements independently to avoid transaction block errors. Operational limitations regarding image filesystem paths were documented.

Feedback provided to Group 5 commended the clean relational schema design and strong integrity constraints enforced across contact fields and reference codes. Recommendations advised implementing an application-level staff authentication table with session tokens to guard administrative routes within local networks, alongside adding unique constraints on match combinations to prevent duplicate entry generation.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 4)

Column Header	Value
Project	Group 5 - PUP-SPOT: Campus Lost and Found System
Repository cloned	https://github.com/grp05-dba/grp05_lostandfound, Branch: main
Database setup result	Success (Initialized PostgreSQL 18.6 on port 5432, loaded 8 tables, categories, demo records, and configured least-privilege roles).
Access-control folder link	https://github.com/Nycri/brewpos/tree/main/access-control
Access-control status	Completed
Feedback sent to owners	Yes (05/11/2026)
PDF progress document	Activity4_Progress_Report_Group5.pdf
Last updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.